Java 对象序列化的机制和原理

目录

1 概述.....2
2 Java 序列化算法透析.....2
2.1 序列化的必要性.....2
2.2 如何序列化一个对象.....2
2.3 对象的序列化格式.....3
2.4 Java 的序列化算法.....4
3 总结.....17
3.1 java 序列化算法的基本步骤.....17
3.2 备注说明.....17

乔治工作室
专注于互联网技术研发 | 让生活更精彩
www.georgeinfo.com

文档名称：	Java 对象序列化的机制和原理
文档整理 补充和修正：	George < www.georgeinfo.com >
参考文章 1：	http://www.java3z.com/cwbwebhome/article/article8/862.html
创建时间：	2015-11-16

1 概述

本文讲解了 Java 序列化的机制和原理。从文中你可以了解如何序列化一个对象，什么时候需要序列化以及 Java 序列化的算法。

有关 Java 对象的序列化和反序列化也算是 Java 基础的一部分，下面对 Java 序列化的机制和原理进行一些介绍。

2 Java 序列化算法透析

序列化 Serialization 是一种将对象以一连串的字节描述的过程。

反序列化 deserialization 是一种将这些字节重建成一个对象的过程。

Java 序列化 API 提供一种处理对象序列化的标准机制。在这里你能学到如何序列化一个对象，什么时候需要序列化以及 Java 序列化的算法，我们用一个实例来示范序列化以后的字节是如何描述一个对象的信息的。

乔治工作室
专注于互联网技术研发，让生活更精彩
www.georgeinfo.com

2.1 序列化的必要性

Java 中，一切都是对象，在分布式环境中经常需要将 Object 从这一端网络或设备传递到另一端。

这就需要有一种可以在两端传输数据的协议，Java 序列化机制就是为了解决这个问题而产生。

2.2 如何序列化一个对象

一个对象能够序列化的前提是实现 Serializable 接口，Serializable 接口没有方法，更像是个标记。

有了这个标记的 Class 就能被序列化机制处理。

```
1.  package test.simpleserial;
2.
3.  import  java.io.Serializable;
4.
5.  class  TestSerial  implements  Serializable  {
6.      public  byte  version  =  100;
7.      public  byte  count  =  0;
```

```
8.  }
```

然后我们写个程序将对象序列化并输出。ObjectOutputStream 能把 Object 输出成 Byte 流。

我们将 Byte 流暂时存储到 temp.out 文件里。

```
1. public static void main(String args[]) throws IOException {
2.   FileOutputStream fos = new FileOutputStream("temp.out");
3.   ObjectOutputStream oos = new ObjectOutputStream(fos);
4.   TestSerial ts = new TestSerial();
5.   oos.writeObject(ts);
6.   oos.flush();
7.   oos.close();
8. }
```

如果要从持久的文件中读取 Bytes 重建对象，我们可以使用 ObjectInputStream。

```
public static void main(String args[]) throws IOException {
    FileInputStream fis = new FileInputStream("temp.out");

    ObjectInputStream oin = new ObjectInputStream(fis);

    TestSerial ts = (TestSerial) oin.readObject();
    System.out.println("version="+ts.version);
}
```

执行结果为：

```
100.
```

2.3 对象的序列化格式

将一个对象序列化后是什么样子呢？打开刚才我们将对象序列化输出的 temp.out 文件，以 16 进制方式显示。内容应该如下：

乔治工作室
专注于互联网技术研发 | 让生活更精彩
www.georgeinfo.com

```
AC ED 00 05 73 72 00 1C 74 65 73 74 2E 73 69 6D
70 6C 65 73 65 72 69 61 6C 2E 54 65 73 74 53 65
72 69 61 6C 78 3D EE 3F 47 F6 3A 07 02 00 02 42
00 05 63 6F 75 6E 74 42 00 07 76 65 72 73 69 6F
6E 78 70 00 64
```

上面这一坨字节，就是用来描述序列化以后的 test.simpleserial.TestSerial 对象的，我们注意到 TestSerial 类中只有两个域(类属性，或者叫类字段，有的资料中，也叫做类的数据成员)：

```
public  byte version = 100;

public  byte count = 0;
```

且都是 byte 型，理论上存储这两个域只需要 2 个 byte，但是实际上 temp.out 占据空间为 69 bytes，也就是说除了数据以外，还包括了对序列化对象的其他描述。

2.4 Java 的序列化算法

序列化算法一般会按步骤做如下事情：

1) 将对象实例相关的类元数据输出。

2) 递归地输出类的超类描述直到不再有超类。

3) 类元数据完了以后，开始从最顶层的超类开始输出对象实例的实际数据值。

4) 从上至下递归输出实例的数据。

我们用另一个更完整覆盖所有可能出现的情况的例子代码，来说明：

```
1. package test.serial;
2.
3. import java.io.FileOutputStream;
4. import java.io.IOException;
5. import java.io.ObjectOutputStream;
6. import java.io.Serializable;
7.
8. class Parent implements Serializable {
```

乔治工作室 专注于互联网技术研发 | 让生活更精彩 www.georgeinfo.com

```
9.      private int  parentVersion  =  10;
10.}
11.
12.class  Contain  implements  Serializable{
13.     private int  containVersion  =  11;
14.}
15.
16.public  class  SerialTest  extends  parent
17.     implements  Serializable  {
18.
19.     private int  version  =  66;
20.     private Contain  con  =  new  contain();
21.
22.     public  int  getVersion()  {
23.         return  version;
24.     }
25.
26.     public  static  void  main(String  args[])
27.         throws  IOException  {
28.       FileOutputStream  fos  =  new  FileOutputStream("temp.out");
29.       ObjectOutputStream  oos  =  new  ObjectOutputStream(fos);
30.       SerialTest  st  =  new  SerialTest();
31.       oos.writeObject(st);
32.       oos.flush();
33.       oos.close();
34.   }
35.}
36.
```

这个例子中，SerialTest 类继承了 Parent 父类，内部还持有一个 Contain 对象属性。

序列化后的格式如下：

乔治工作室
专注于互联网技术研发 | 让生活更精彩
www.georgeinfo.com

AC ED 00 05 73 72 00 16 74 65 73 74 2E 73 65 72

69 61 6C 2E 53 65 72 69 61 6C 54 65 73 74 86 0C

7A 8A 40 FC C0 C9 02 00 02 49 00 07 76 65 72 73

69 6F 6E 4C 00 03 63 6F 6E 74 00 15 4C 74 65 73

74 2F 73 65 72 69 61 6C 2F 43 6F 6E 74 61 69 6E

3B 78 72 00 12 74 65 73 74 2E 73 65 72 69 61 6C

2E 50 61 72 65 6E 74 C4 39 29 89 B9 0A 65 A9 02

00 01 49 00 0D 70 61 72 65 6E 74 56 65 72 73 69

6F 6E 78 70 00 00 00 0A 00 00 00 42 73 72 00 13

74 65 73 74 2E 73 65 72 69 61 6C 2E 43 6F 6E 74

61 69 6E FC F8 DC 6B DB 14 FC EB 02 00 01 49 00

0E 63 6F 6E 74 61 69 6E 56 65 72 73 69 6F 6E 78

70 00 00 00 0B

我们来仔细看看这些字节(下文中的字节数据，使用十六进制数字表示一个字节，省略了十六进制数前面的 0x 字面量前缀，如 0xAC，直接写伪：AC)的含义。

开头部分，见黄色标注：

- ***AC ED：STREAM_MAGIC.　声明使用了序列化协议***

对应的值是十进制数字 172，也就是
java.io.ObjectStreamConstants.STREAM_MAGIC
这个常量对应的值。

- ***00 05：STREAM_VERSION.　序列化协议版本.***

对应的值是十进制数字 5，也就是
java.io.ObjectStreamConstants.STREAM_VERSION
这个常量对应的值。

接下来，开始输出直接被序列化的类(以下简称“直接类”，是相对于“直接类”的父类，和“直接类”中所包含的对象型属性的类而言)的描述。

- ***73：TC_OBJECT.　声明这是一个新的对象.***

对应的值是十进制数字 115，也就是
java.io.ObjectStreamConstants.TC_OBJECT
这个常量对应的值。

乔治工作室
专注于互联网技术研发 | 让生活更精彩
GEORGE STUDIO

序列化算法的第一步就是输出对象相关类的描述。例子所示对象为 SerialTest 类实例，因此接下来输出 SerialTest 类的描述。见绿色标注：

- ***72：TC_CLASSDESC.　声明这里开始一个新类(开始一个新 CLASS 描述)。***

对应的值是十进制数字 114，也就是
java.io.ObjectStreamConstants.TC_CLASSDESC
这个常量对应的值。

- ***00 16：直接类的名字的长度.***

*类名字的长度（**包括包路径**）。*
在本例中，class 的名字是“test.serial.SerialTest”，所以其长度是 0x16，也就是类名字占用类名长度之后的 22 个字节。

- **74 65 73 74 2E 73 65 72 69 61 6C 2E 53 65 72 69 61 6C 54 65 73 74: 直接类的类名子.**

在本例中，就是字符串“test.serial.SerialTest”，字符串编码是根据类文件创建时的 Java 文件编码来的，一般还原类名时，直接使用 new String(byte[])还原类名即可。

在本例中，使用

```
String className
= new String(new byte[]{0x74, 0x65, 0x73, 0x74, 0x2E, 0x73,
0x65, 0x72, 0x69, 0x61, 0x6C, 0x2E, 0x53, 0x65, 0x72, 0x69, 0x61, 0x6C
, 0x54, 0x65, 0x73,  0x74});
```

即可还原出类名字字符串：test.serial.SerialTest
如果你清楚地知道当初.java 文件创建时的编码，比如是 gbk 编码，那么还原类名字时，就可以指定 String 的编码了。

- **86 0C 7A 8A 40 FC C0 C9: SERIALVERSIONUID, 序列化 ID，如果没有指定，则会由算法随机生成一个 8BYTE 的 ID.**

乔治工作室
专注于互联网技术研发 | 让生活更精彩
GEORGE STUDIO
make life wonderful
www.georgeinfo.com

创建类的时候，可以使用：

```
private static final long serialVersionUID
                                = 比较大的长整形数字 L;
```

的方式，指定类的序列化 ID。
本例中，86 0C 7A 8A 40 FC C0 C9 对应的值，就是这个类随机生成的 serialVersionUID，对应的十进制数字是：575525617664
序列化 ID 的类型为 long 型因此占用 8 个字节，使用如下代码获得 86 0C 7A 8A 40 FC C0 C9 对应的 long 值：

```
byte[] svidBytes = new byte[64];
System.arraycopy(intToByteArray(0x86), 0, svidBytes, 0, 4);
System.arraycopy(intToByteArray(0x0C), 0, svidBytes, 4, 1);
System.arraycopy(intToByteArray(0x7A), 0, svidBytes, 5, 1);
System.arraycopy(intToByteArray(0x8A), 0, svidBytes, 6, 4);
System.arraycopy(intToByteArray(0x40), 0, svidBytes, 10, 1);
System.arraycopy(intToByteArray(0xFC), 0, svidBytes, 11, 4);
System.arraycopy(intToByteArray(0xC0), 0, svidBytes, 15, 4);
System.arraycopy(intToByteArray(0xC9), 0, svidBytes, 19, 4);

long svid = bytes2Long(svidBytes);
System.out.println(svid);
```

```
    /**
     * 高效的 int -> byte[]转化
     */
    public static byte[] intToByteArray(final int integer) {
        int byteNum = (40 - Integer.numberOfLeadingZeros(integer < 0 ?
~integer : integer)) / 8;
        byte[] byteArray = new byte[4];

        for (int n = 0; n < byteNum; n++) {
            byteArray[3 - n] = (byte) (integer >>> (n * 8));
        }

        return (byteArray);
    }

    /**
     * 高效 byte[] -> long 工具方法
     */
    public static long bytes2Long(byte[] byteNum) {
        long num = 0;
        for (int ix = 0; ix < 8; ++ix) {
            num <<= 8;
            num |= (byteNum[ix] & 0xff);
        }
        return num;
    }
```

工作室
研发 | 让生活更精彩
www.georgeinfo.com

02: 标记号. 该值声明该对象支持序列化。

该字节的 8 位分表代表不同的含义，SC_EXTERNALIZABLE 0x04 ：该类实现了 java.io.Externalizable 接口 SC_BLOCK_DATA 0x08 ：Externalizable 接口的 writeExternal 方法写入的数据 SC_SERIALIZABLE 0x02 ：该类实现了 java.io.Serializable 接口 SC_WRITE_METHOD 0x01 ：该序列化类实现了 writeObject 方法 SC_ENUM 0x10 ：该类是枚举(enum)类型该标记号通过上述信息进行或运算（|）而获得。

00 02: 该类所包含的域个数。

指该类所包含的类属性(也叫做类字段，或类的数据成员)的数量（不包括父类中的域数量），在本例中，SerialTest 类的域(类属性)个数，只有 2 个，分别是：

```
private int version = 66;
```

```
private contain con = new contain();
```

所以，这两个字节是 0x00 0x02，转换成十进制数字是 2，也就是该类直属类域成员，只有 2 个。

接下来，算法输出其中的一个域 int version = 66 的域描述，见浅蓝色标注：

- ***49： 域类型. 49 代表"I"， 也就是 INT.***

  0x49，转换成十进制数字是：73，ascii 码 73 对应的字符是大写字母 I，使用 System.out.println((char)73);可以打印出字符 I

- ***00 07： 域名字的长度.***

  在本例中，第一个域(version)名字的长度，version=7 个字符

- ***76 65 72 73 69 6F 6E：第一个域名字.***

  在本例中，第一个域名字是“version”，使用如下代码：

```
byte[] ba = new byte[]{0x76, 0x65, 0x72, 0x73, 0x69, 0x6F, 0x6E};
System.out.println(new String(ba));
```

可以打印出 0x76, 0x65, 0x72, 0x73, 0x69, 0x6F, 0x6E 这 7 个字节，对应的字符串，就是本类的第一个域的名字：version

然后，算法输出下一个域，contain con = new contain();这个有点特殊，是个对象。

描述对象类型引用时需要使用 JVM 的标准对象签名表示法，见灰色标注：

- ***4C： 域的类型.***

```
private Contain con = new Contain();
```

这个域，是一个对象类型的属性域，使用字符 L 表示，0x4C 转换为十进制数字是 76，ascii 码 76 对应的字符是 L。

至于为什么用 L 来表示对象类型，我也不清楚。

乔治工作室
专注于互联网技术研发 | 让生活更精彩
www.georgeinfo.com

- ***00 03:　域名字长度.***

  在本例中，第二个域名字是 con，所以其长度是 0x03，也就是十进制数字：3 个字符的长度。

- ***63 6F 6E:　直接类的第二个域的名字***

  在本例中，就是字符串：con

- ***74: TC_STRING.　代表一个 NEW STRING 开始***

  java.io.ObjectStreamConstants.TC_STRING

  对应的值：0x74

- ***00 15:　第二个域类型的 JVM 标准对象签名的字节长度.***

  本例 中，第二个域的 JVM 标准对象签名是 Ltest.serial.Contain;

  注意：包括前面的大写 L 和最后的分号;

  这个字符串的长度是 21 个字节，所以用十六进制数字：0x15 来表示。

- ***4C 74 65 73 74 2F 73 65 72 69 61 6C 2F 43 6F 6E 74 61 69 6E 3B:　第二个域的 JVM 标准对象签名：LTEST.SERIAL.CONTAIN;.***

  在本例中，第二个域是对象 test.serial.Contain，它用 JVM 标准对象签名表示是：Ltest.serial.Contain;，这个字符串对应的字节是：4C 74 65 73 74 2F 73 65 72 69 61 6C 2F 43 6F 6E 74 61 69 6E 3B，占用 21 个字节。

  (如果第二个域是 String 类型，那么它的 JVM 标准对象签名就是：Ljava.lang.String;)

  对象签名包括前面的大写 L 和最后的分号;

- ***78: TC_ENDBLOCKDATA,对象数据块结束的标志.***

乔治工作室
GEORGE STUDIO
make life wonderful

java.io.ObjectStreamConstants.TC_ENDBLOCKDATA

对应的值：0x78

接下来，输出直接类的父类，也就是 test.serial.Parent 类描述了，见红色标注：

- ***72：TC_CLASSDESC.　声明开始一个新类的描述.***

java.io.ObjectStreamConstants. TC_CLASSDESC

对应的值：0x72

- ***00 12：　父类类名长度.***

在本例中，父类名是“test.serial.Parent”（类名指包括包路径的全名，本例中类没有包路径），父类类名“test.serial.Parent”占 18 个字节的长度，所以用 0x12 来表示。

- ***74 65 73 74 2E 73 65 72 69 61 6C 2E 50 61 72 65 6E 74:父类类名字：　TEST.SERIAL.PARENT***

在本例 中，父类的名字是：test.serial.Parent

```
byte[] ba = new byte[]{0x74, 0x65, 0x73, 0x74, 0x2E, 0x73, 0x65, 0x72,
0x69, 0x61, 0x6C, 0x2E, 0x50, 0x61, 0x72, 0x65, 0x6E, 0x74};
System.out.println(new String(ba));
```

可以打印出父类的名字。

- ***C4 39 29 89 B9 0A 65 A9：　SERIALVERSIONUID,　父类 PARENT 的序列化 ID.***

父类 parent 的序列化版本 ID，长整形，8 个字节的长度。

乔治工作室 GEORGE STUDIO make life wonderful 专注于互联网技术研发 构建生活更精彩 www.georgeinfo.com

- ***02: 标记号. 该值声明该对象支持序列化.***

  具体含义见上面对直接子类 SerialTest 的标记号分析。

- ***00 01: 父类中域的个数.***

  在本例中，父类只有一个域：

  ```
  int parentVersion = 10;
  ```

  所以这里是 0x01

下一步，输出 parent 类的域描述：

```
private int parentVersion = 100;
```

同见红色标记：

- ***49: 域类型. 49 代表"I", 也就是 INT.***

  0x49，转换成十进制数字是：73，ascii 码 73 对应的字符是大写字母 I，使用 System.out.println((char)73);可以打印出字符 I

- ***00 0D: 父类域名字长度.***

  本例中，父类第一个域名字是：parentVersion，也就是 13 个字符(英文是 ascii 字符，所以一个字符占用一个字节的长度)，所以长度是：0x0D = 十进制数 13

- ***70 61 72 65 6E 74 56 65 72 73 69 6F 6E: 父类第一个域的名字。***

  在本例中，是字符串：parentVersion

- ***78: TC_ENDBLOCKDATA,对象块结束的标志。***

  ```
  java.io.ObjectStreamConstants. TC_ENDBLOCKDATA = 0x78
  ```

乔治工作室 GEORGE STUDIO make life wonderful 生活更精彩 www.georgeinfo.com

- ***70：TC_NULL，　说明没有其他父类的标志。.***

  java.io.ObjectStreamConstants.　TC_ENDBLOCKDATA　= 0x70

到此为止，算法已经对所有的类的描述都做了输出。下一步就是把实例对象的实际值输出了。这时候是从 parent Class 的域开始的，见深绿色标注：

- ***00　00　00　0A：父类 PARENTVERSION 域的值=10.***

  在本例中，父类的 parentVersion 域的值是 int 整形：10，所以这里存储 0x0A，并且占用 4 个字节（因为 int 占用 4 个字节）

接下来，输出子类 SerialTest 的各个域的值：

- ***00　00　00　42：子类的 VERSION 域的值=66.***

  子类的第一个域 int version = 66;

  所以其值是 66，十六进制 0x42=十进制数字 66

  因为 int 类型占 4 个字节，所以是 0x00　0x00　0x00　0x42

再往后的 bytes 比较有意思，算法需要描述 contain 类的信息，要记住，现在还没有对 contain 类进行过描述，见绿色标注：

- ***73：TC_OBJECT，　声明这是一个新的对象.***

  对应的值是十进制数字 115，也就是
  java.io.ObjectStreamConstants.TC_OBJECT
  这个常量对应的值。

- ***72：TC_CLASSDESC 声明这里开始一个新 CLASS.***

  对应的值是十进制数字 114，也就是
  java.io.ObjectStreamConstants.TC_CLASSDESC
  这个常量对应的值。

- ***00　13：　属性类的类名的长度.***

  在本例中，属性类 test.serial.Contain 的类名字(包括包路径的全类名)

赤沿工作室
专注于互联网技术研发 | 让生活更精彩
www.geinfo.com

是 19 个 ascii 字符，所以占用 19 个字节，所以这里是 0x13，换算成十进制是数字：19

- **74 65 73 74 2E 73 65 72 69 61 6C 2E 43 6F 6E 74 61 69 6E:属性类 CONTAIN 的类全名= TEST.SERIAL.CONTAIN**

使用如下代码：

```
byte[] ba = new
byte[]{0x74,0x65,0x73,0x74,0x2E,0x73,0x65,0x72,0x69,0x61,0x6C,
0x2E,0x43,0x6F,0x6E,0x74,0x61,0x69,0x6E};
```

```
System.out.println(new String(ba));
```

可以输出该属性类的类全名是：test.serial.Contain

- **FC F8 DC 6B DB 14 FC EB: SERIALVERSIONUID, 属性类的序列化 ID.**

同上文中的 serialVersionUID 解释。

因为 serialVersionUID 是 long 类型，占用 8 个字节，所以这里是取 8 个字节的长度。

乔治工作室
专注于互联网技术研发 | 让生活更精彩
www.georgeinfo.com

- **02:标记号. 该值声明该对象支持序列化**

该字节的 8 位分表代表不同的含义，SC_EXTERNALIZABLE 0x04 ：该类实现了 java.io.Externalizable 接口 SC_BLOCK_DATA 0x08 ：Externalizable 接口的 writeExternal 方法写入的数据 SC_SERIALIZABLE 0x02 ：该类实现了 java.io.Serializable 接口 SC_WRITE_METHOD 0x01 ：该序列化类实现了 writeObject 方法 SC_ENUM 0x10 ：该类是枚举(enum)类型该标记号通过上述信息进行或运算（|）而获得。

- **00 01: 类内的域个数。**

本例中，属性类 test.serial.Contain 中只有一个属性：

```
int containVersion = 11;
```

所以属性类的域个数时 1

接着，输出属性类 *test.serial.Contain* 的唯一一个属性域的描述：

```
int containVersion = 11;
```

- ***49: 域类型. 49 代表"I", 也就是 INT.***

  解释同上文

- ***00 0E: 域名字长度.***

  本例中域是 containVersion，所以其名字长度是：14dec=0x0E

- ***63 6F 6E 74 61 69 6E 56 65 72 73 69 6F 6E: 域名字 = CONTAINVERSION.***

  本例中，域名字是 containVersion，所以：

  "containVersion" = new String(new byte[]{0x63, 0x6F, 0x6E, 0x74, 0x61, 0x69, 0x6E, 0x56, 0x65, 0x72, 0x73, 0x69, 0x6F, 0x6E});

专注于互联网技术研发 | 让生活更精彩
www.georgeinfo.com

- ***78: TC_ENDBLOCKDATA 对象块结束的标志.***

  java.io.ObjectStreamConstants.TC_ENDBLOCKDATA

  对应的值：0x78

这时，序列化算法会检查 contain 是否有超类，如果有的话会接着输出。

- ***70:TC_NULL，没有超类了。***

  java.io.ObjectStreamConstants. TC_ENDBLOCKDATA = 0x70

最后，将 contain 类实际域值输出。

- ***00 00 00 0B: 属性类的属性域 CONTAINVERSION 的值 =11.***

本例中，

```
int containVersion = 11;
```

int 类型占用 4 个字节，所以用 0x00 0x00 0x00 0x0B 来表示数字 11

至此，整个示例的二进制字节数组讲解完毕。

3 总结

3.1 java 序列化算法的基本步骤

1) 输出序列化的头部信息，包括标识序列化协议的幻数以及协议的版本；
2) 按照由子类到父类的顺序，递归的输出类的描述信息，直到不再有父类为止；类描述信息按照类元数据，类属性信息的顺序写入序列化流中；
3) 按照由父类到子类的顺序，递归的输出对象域的实际数据值；而对象的属性信息是按照基本数据类型到 java 对象类型的顺序写入序列化流中；其中 java 对象类型的属性会从步骤 a)重新开始递归的输出，直到不再存在 java 对象类型的属性。

本文从 JDK 源代码的角度分析了 java 默认的序列化机制的基本原理，最后总结出了序列化的算法。但是这种序列化是不安全，因为序列化二进制格式完全编写在文档中或者在网络中传播，并且完全可逆。如果采用 java 默认序列化机制在网络上传输商业机密，我们完全可以截获这些二进制数据按照上述规则获取这些机密信息。

为了解决这一问题 java 提供了自定义的序列化的方法。可以通过在序列化类中添加 readObject 和 writeObject 方法以及实现 java.io.Externalizable 接口，实现自定义序列化的实现。

3.2 备注说明

本文主要以

http://www.java3z.com/cwbwebhome/article/article8/862.html

这篇文章作为参考，但是这篇文章中，所使用的例子代码不好，因为类名字手写字母不是大写，并且类也没有包路径，很容易给人错误的指引，让人以为序列化后的字节码中，类名字是不包含包路径的，实际上是包含包路径的全类名。

所以本文档对上述引用文章的例子代码，进行了重新改写，以便适用最典型的场景。

另外，http://jianleixing.iteye.com/blog/2015495 这篇文章，对 JVM 默认序列化对象的相关 JDK 源代码进行了分析，但是原作者申明不允许转载，所以本文档中并没有介绍该文章所列举的 JDK 源码片段。

乔治工作室
专注于互联网技术研发 | 让生活更精彩
www.georgeinfo.com
GEORGE STUDIO
make life wonderful